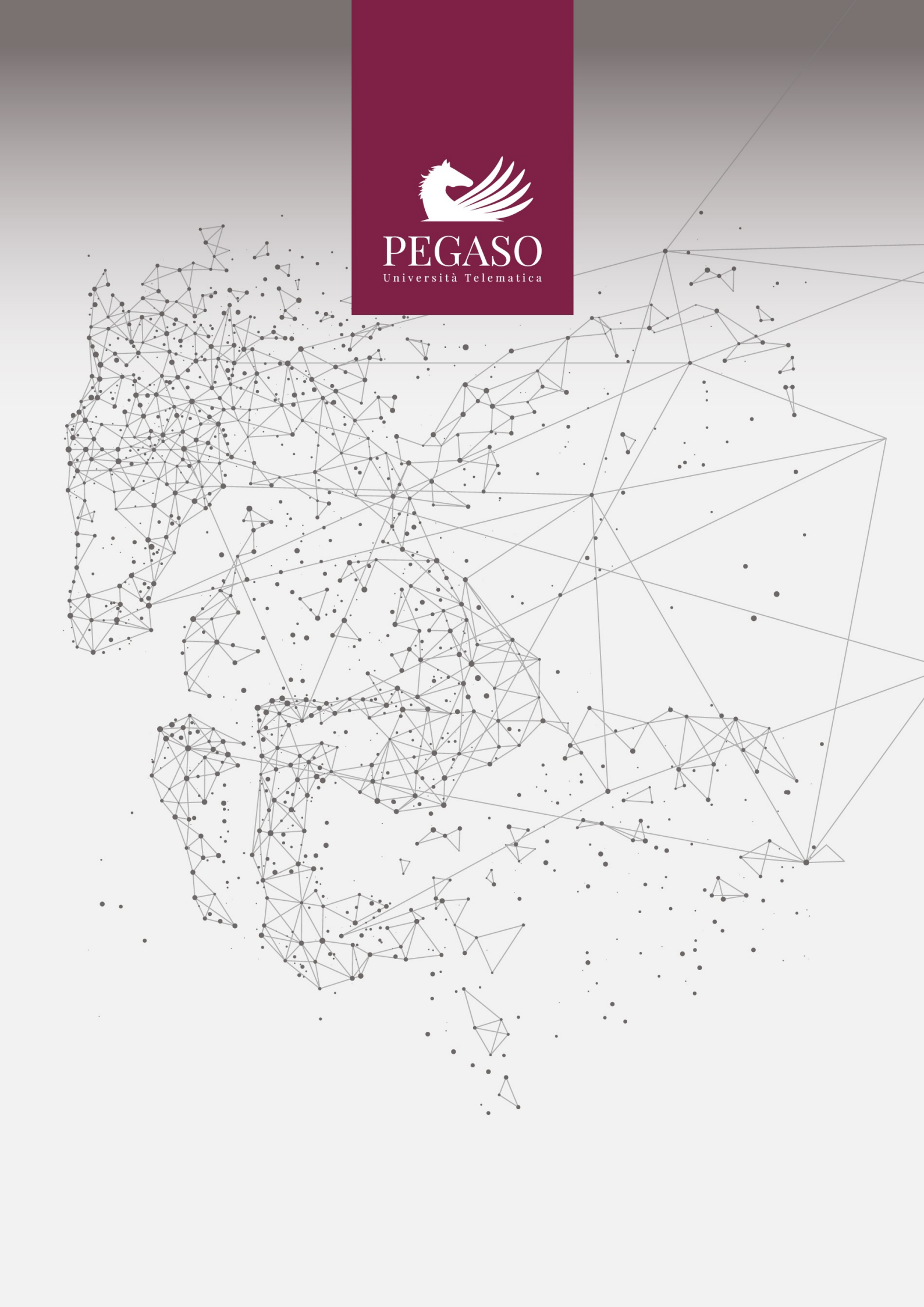

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. FILTRI E RAGGRUPPAMENTI....4
3. PARTIZIONARE DATI....7
4. INTERAZIONI CON I FILE DI TESTO....11
5. ESEMPIO DI ANALISI DI DATI DA CSV....12
6. DATAFRAME....15
BIBLIOGRAFIA....25

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presenta una serie di operazioni che coinvolgono le strutture RDD. In particolare, vengono introdotti i filtri e i raggruppamenti e le interazioni del RDD con i file di testo. Si illustrano anche la partizione dei dati e le funzioni che ne permettono il controllo. Viene, inoltre, presentato un esercizio di analisi di dati da CSV con l'uso di RDD. Infine, viene presentata un'altra struttura dati di PySpark, il DataFrame, una collezione immutabile di dati elaborabile in parallelo su più nodi di un cluster, simile dal punto di vista di alcune funzionalità ai DataFrame Pandas.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Filtri e raggruppamenti

Tra le operazioni più importanti in assoluto in PySpark troviamo **l'esecuzione di filtri e raggruppamenti** che permettono di eseguire selezioni ed estrarre risultati dai dati a disposizione.

Ad esempio, se volessimo recuperare le insufficienze presenti in *rdd_voti*, potremmo utilizzare il metodo **filter**() e una funzione al suo interno che esprime il criterio da applicare come mostrato nel codice seguente.

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd_voti = sc.parallelize([6,8,8,9,5,6,6,8])
filtered_rdd_voti = rdd_voti.filter(lambda voto: voto<6)
ris = filtered_rdd_voti.collect()

print(ris)
sc.stop()
```

```
[5]
```

Attraverso il codice precedente, viene restituita una lista contenente un singolo elemento, pari a 5, che è l'unico voto presente nella lista inferiore a 6. È stato possibile utilizzare un ***collect***() per estrarre il risultato in quanto un filtro genera, a sua volta, un RDD.

Infatti, è possibile utilizzare qualsiasi operazione tra quelle viste finora a valle di un **filter**(). Ad esempio, per un conteggio al fine di sapere quanti sono gli 8 presenti nella lista, il codice precedente deve essere modificato come segue.

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd_voti = sc.parallelize([6,8,8,9,5,6,6,8])
filtered_rdd_voti = rdd_voti.filter(lambda voto: voto==8)
ris = filtered_rdd_voti.collect()
num_8 = filtered_rdd_voti.count()

print(ris)
print(num_8)
sc.stop()
```

```
[8, 8, 8]
3
```

Il **raggruppamento** viene svolto in generale in almeno due fasi:

1. si esprime con una **funzione** il **criterio per il raggruppamento**. Se la funzione è sufficientemente sintetica, si opta in genere per una lambda;

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. sui valori inseriti in ogni gruppo si può applicare una funzione fornita con il metodo **mapValues()**. Anche in questo caso può essere una funzione creata da noi, una funzione già presente in Python o un'espressione lambda.

Una volta elaborati i valori del **groupBy**(), si può passare alle esportazioni del risultato, ad esempio, con un **collect**().

Si fornisce, ora, un esempio per chiarire quanto affermato. Si suddividono i valori in due gruppi:

1. tutti i valori **sufficienti** da una parte etichettati complessivamente con la stringa SUF;
2. tutti i valori **insufficienti** dall'altra etichettati dalla stringa INSUF.

I valori dei voti sono quelli presenti in *rdd_voti* ovvero:

6,8,8,9,5,6,6,8

Nel codice seguente, per il **raggruppamento** al metodo groupBy() viene passata la funzione lambda lambda voto:'SUF' if voto>=6 else 'INSUF'

Affinché il risultato del **groupBy()** sia fruibile, deve essere passato per **mapValues()** che applica una trasformazione, mediante funzione, a tutti valori di ogni gruppo. In questo caso, è stata applicata ***list()***, cioè la funzione che fa da costruttore alle liste Python.

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd_voti = sc.parallelize([6,8,8,9,5,6,6,8])
grouped_rdd_voti = rdd_voti.groupBy(lambda voto:
                                    'SUF' if voto>=6 else 'INSUF')
grouped_list = grouped_rdd_voti.mapValues(list)
ris = grouped_list.collect()

print(ris)
sc.stop()
```

```
[('SUF', [6, 8, 8, 9, 6, 6, 8]), ('INSUF', [5])]
```

Al termine si ottiene una **lista Python** contenente **due elementi** costituiti ognuno da una **tupla di due valori.**

Ogni tupla all'interno contiene:

- la **stringa** con il ruolo di **etichetta**;
- la **sequenza di voti** che le corrispondono.

In sintesi:

('INSUF',[5])

dice che c'è solo un voto insufficiente (il 5) e viene restituito sotto forma di lista, mentre

('SUF',[6, 8, 8, 9, 6, 6, 8])

elenca i voti sufficienti che sono in numero maggiore e inclusi anch'essi in una lista.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Con lo script seguente, invece, si ottiene il voto massimo per ogni gruppo usando la **funzione built-in** di Python **max()**.

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd_voti = sc.parallelize([6,8,8,9,5,6,6,8])
grouped_rdd_voti = rdd_voti.groupBy(lambda voto:
                                    'SUF' if voto>=6 else 'INSUF')
grouped_max = grouped_rdd_voti.mapValues(max)
ris = grouped_max.collect()

print(ris)
sc.stop()
```

```
[('SUF', 9), ('INSUF', 5)]
```

Con lo script seguente si usa un'espressione lambda per ottenere la **definizione del range di valori** indicato dai voti massimo e minimo di ogni gruppo all'interno del **mapValues**().

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd_voti = sc.parallelize([6,8,8,9,5,6,6,8])
grouped_rdd_voti = rdd_voti.groupBy(lambda voto:
                                    'SUF' if voto>=6 else 'INSUF')
grouped_range = grouped_rdd_voti.mapValues(lambda voti: (min(voti), max(voti)))
ris = grouped_range.collect()

print(ris)
sc.stop()
```

```
[('SUF', (6, 9)), ('INSUF', (5, 5))]
```

Con i risultati del **collect**() è possibile «muoversi» all'interno delle strutture dati considerando che sia le tuple, sia le liste sono navigabili in base alla posizione.

```
grouped_range = grouped_rdd_voti.mapValues(lambda voti: (min(voti), max(voti)))
sorted_grouped_range = grouped_range.sortBy(lambda x: x, ascending = True)

estremi = sorted_grouped_range.collect()

print(estremi)
print(estremi[1][1][1])
sc.stop()
```

```
[('INSUF', (5, 5)), ('SUF', (6, 9))]
9
```

Con i tre indici ad uno [1] utilizzati per indicizzare la lista *estremi* si indicano, rispettivamente:

1. il secondo elemento della lista relativo ai voti sufficienti;
2. la tupla con i due estremi (6, 9);
3. il secondo numero che rappresenta il loro massimo ovvero 9.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Partizionare dati

Uno degli aspetti caratterizzanti di Spark è la sua **capacità di parallelizzare il lavoro su più nodi di un cluster**. Al fine di agevolare Spark tale parallelizzazione, è essenziale comprendere il **concetto di partizione**.

Una ***partizione*** è una **suddivisione logica di un grande dataset**. Suddividere i dati in più partizioni permette la distribuzione della loro elaborazione su vari nodi.

Per sfruttare al massimo le potenzialità di un cluster Spark si dovrebbero avere almeno tante **partizioni** quanti sono i **core** a disposizione tra i vari nodi. Ciò permette di **migliorare le prestazioni** che su grandi quantità di dati rischiano di essere pesantemente penalizzate.

Ad ogni modo, è necessario considerare che l'impiego di partizioni **non sempre garantisce il successo per l'ottimizzazione**. Tali partizioni andrebbero calibrate, infatti, in base alla tipologia di dati e agli obiettivi dell'analisi.

Se un maggior numero di partizioni permette di attivare più nodi, un loro minore numero manda in elaborazione blocchi di dati più grandi accelerando spesso le operazioni.

Da un punto di vista pratico, ogni **RDD dispone del necessario per la gestione delle partizioni**. Gli elementi di base sono, infatti:

- Il metodo **getNumPartitions**() che fornisce il numero di partizioni in cui una RDD è suddivisa in quel momento. Anche nei casi in cui non si effettuano partizioni, Spark prende comunque in considerazione la problematica scegliendo autonomamente il numero di partizioni per noi in base:
  - alle caratteristiche del sistema;
  - alla configurazione dello *SparkContext.*
- Il **partitioner**(), cioè una funzione che sceglie il **criterio in base al quale partizionare i dati**. Di default, Spark ne mette a disposizione uno che semplicemente separa i dati in base alla loro quantità. È possibile, tuttavia, creare uno specifico criterio personalizzato.

- Altri metodi che permettono sia la **definizione di partizioni**, sia il loro **raggruppamento**:
  - **partitionBy**() che permette di ripartire i dati in partizioni;
  - **glom**() che li riaggrega in un unico RDD;
  - **coalesce**() che riduce il numero di partizioni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Quando si utilizza il **partitionBy()** è necessario specificare un primo argomento, obbligatorio, di tipo intero, che imposta il **numero di partizioni desiderate**. Eventualmente un secondo argomento che rappresenta la nuova funzione **partitioner**.

Per comprendere come funziona la **gestione delle partizioni** in modo semplice possiamo utilizzare un comune elenco di numeri come il seguente:

```
rdd = sc.parallelize(range(1,26))
```

Una normale azione di **parallelize()** crea un RDD e con un **getNumPartitions()** possiamo conoscere il numero delle partizioni in cui è suddiviso mentre il **partitioner** non è impostato (riferimento a None) come mostrato nel codice seguente.

```
rdd = sc.parallelize(range(1,26))
print(rdd.getNumPartitions())
```

```
1
```

Se si vogliono adottare delle partizioni, definite in automatico da Spark, se ne può passare il numero come secondo argomento di **parallelize**(). Nel caso seguente, il range di 25 valori sarà già ripartito in quattro settori (4 passato come secondo parametro di parallelize()), dimostrato da **getNumPartitions**():

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd = sc.parallelize(range(1,26),4)
print(rdd.getNumPartitions())

sc.stop()
```

```
4
```

Il metodo **glom**() può essere usato per riaggregare le partizioni in un'unica struttura.

Presentandole con **collect**() vediamo infatti come le parentesi quadre suddividano i numeri quattro zone:

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd = sc.parallelize(range(1,26),4)
print(rdd.glom().collect())

sc.stop()
```

```
[[1, 2, 3, 4, 5, 6], [7, 8, 9, 10, 11, 12],
[13, 14, 15, 16, 17, 18], [19, 20, 21, 22, 2
3, 24, 25]]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ogni lista tra parentesi quadre è la **trasposizione di una partizione**. Si sottolinea che le partizioni appena viste sono il frutto di una suddivisione di Spark **in base al partitioner standard**.

Infatti, i numeri collocati nelle varie partizioni sono perfettamente in ordine.

È possibile, però, **creare partizioni personalizzate** e per ottenerle dobbiamo utilizzare **partitionBy**(). Prima dell'applicazione del **partitionBy**(), però, vanno organizzati i dati con un *map* in modo tale che ogni valore sia accoppiato con una chiave numerica che, in questo caso, si sceglie come resto della divisione tra il numero stesso e 7 (si sottolinea che tale scelta è stata effettuata a puro scopo esemplificativo). Analizzando il risultato con **glom**():

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd = sc.parallelize(range(1,26))
rdd_part=rdd.map(lambda x: (x%7,x)).partitionBy(6)
print(rdd_part.glom().collect())

sc.stop()
```

```
[[(6, 6), (0, 7), (6, 13), (0, 14), (6, 20),
(0, 21)], [(1, 1), (1, 8), (1, 15), (1, 2
2)], [(2, 2), (2, 9), (2, 16), (2, 23)],
[(3, 3), (3, 10), (3, 17), (3, 24)], [(4,
4), (4, 11), (4, 18), (4, 25)], [(5, 5), (5,
12), (5, 19)]]
```

si può osservare che i gruppi disegnati in base all'operazione *x%7* sono 7 (i possibili resti, infatti, vanno da 0 a 6). Sono state però richieste 6 partizioni. È per questo che quelle con etichette 6 e 0 sono state accorpate (si veda la prima riga).

Si possono condurre ulteriori esperimenti utilizzando **operazioni diverse** nel *map* per **generare la chiave di partizionamento** e osservare mediante **glom**() come vengono ripartite. Con il metodo **coalesce**() si può procedere alla riduzione delle partizioni. Per esempio, le partizioni precedenti vengono ridotte a tre nel seguente modo:

```
rdd = sc.parallelize(range(1,26))
rdd_part=rdd.map(lambda x: (x%7,x)).partitionBy(6)
rdd_unite=rdd_part.coalesce(3)
```

Osservando ancora il risultato di **glom**() potremmo vedere la loro ripartizione:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')

rdd = sc.parallelize(range(1,26))
rdd_part=rdd.map(lambda x: (x%7,x)).partitionBy(6)
rdd_unite=rdd_part.coalesce(3)
print(rdd_unite.glom().collect())

sc.stop()
```

```
[[(6, 6), (0, 7), (6, 13), (0, 14), (6, 20),
(0, 21), (1, 1), (1, 8), (1, 15), (1, 22)],
[(2, 2), (2, 9), (2, 16), (2, 23), (3, 3),
(3, 10), (3, 17), (3, 24)], [(4, 4), (4, 1
1), (4, 18), (4, 25), (5, 5), (5, 12), (5, 1
9)]]
```

Anche **getNumPartitions**() confermerà la strutturazione di dati su tre partizioni come si può osservare dal codice seguente:

```
print(rdd_unite.getNumPartitions())
```

```
3
```

Si noti che anche **coalesce**() ha proceduto **aggregando per chiavi di partizionamento**, infatti, valori con medesima chiave sono finiti nella stessa partizione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Interazioni con i file di testo

I dati su file possono essere recuperati con il metodo **textFile**() invocato su *SparkContext* come riportato nel codice seguente in cui il file di testo *dati.txt* contiene una lista di numeri separati da virgola.

```
from pyspark import SparkContext

sc = SparkContext('local','Esempio')
rdd = sc.textFile('dati.txt')
print(rdd.collect())

sc.stop()
```

```
['21,95,45,34,57,25,88,6,9,18,25,34']
```

L'argomento che si passa come argomento alla funzione **textFile**(), infatti, può rappresentare l'indirizzo di un file o di una cartella. Nel caso di una cartella (quindi stringa senza estensione di file), tutto il contenuto dei file in essa contenuti sarà veicolato mediante un **unico RDD**. Inoltre, conoscendo l'indirizzo del ***NameNode*** è possibile **accedere anche a risorse su HDFS** con il seguente codice di esempio:

```
rdd=sc.textFile('hdfs://localhost:9000/archivio/dati.txt')
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Esempio di analisi di dati da CSV

Si propone ora un esempio riepilogativo di un'analisi di dati eseguita con PySpark. Si utilizzerà un file dalle dimensioni ridotte (poco più di 5.000 righe), ma proponendo approcci replicabili, con lo stesso codice, su **quantità di dati classificabili come Big Data**.

Il dataset che verrà utilizzato è composto da righe ognuna delle quali rappresenta un'operazione su un conto corrente bancario. Il formato di ogni riga è il seguente:

2009-12-28,-155,PAGO,400403,IT001

dove:

- *2009-12-28* è la data dell'operazione;
- *-155* rappresenta l'importo della transazione;
- *PAGO* è un codice alfabetico che rappresenta la tipologia di operazione (quattro possibili codici PAGO per pagamento con bancomat, BODA per bonifico ricevuto, BNFC per bonifico eseguito e VERS per il versamento in contanti);
- *400403* è il numero del conto;
- *IT001* è la filiale di appartenenza del conto.

Gli obiettivi dell'esercizio sono i seguenti:

1. caricare i dati all'interno di un RDD;
2. conteggiare il totale di record (numero di operazioni totali);
3. eseguire un conteggio di singoli record suddividendoli per tipologia;
4. calcolare il saldo di ogni conto corrente suddiviso per anni.

Obiettivo 1-2: caricare i dati all'interno di un RDD e conteggiare il totale di record.

Per il caricamento dei dati, considerando che il file prende il nome di *dati_bancari_annuali.csv*, si procede con il metodo *textFile()*. Verifichiamo dal risultato che sono stati letti 5.146 elementi.

```
lista=sc.textFile('dati_bancari_annuali.csv').collect()
print(len(lista))
```

```
5146
```

Obiettivo 3: eseguire un conteggio di singoli record suddividendoli per tipologia.

In un modulo Python si definiscono due funzioni che verranno utilizzate nelle operazioni di mapping sui dati *row_parsing*() e *row_parsing_ext*():

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def row_parsing(r):
  splitted=r.split(',')
  return (splitted[2],float(splitted[1]))

def row_parsing_ext(r):
  splitted=r.split(',')
  from datetime import datetime as dt
  year=dt.strptime(splitted[0],'%Y-%m-%d').year
  return ((year,splitted[3]), float(splitted[1]))
```

Entrambe le funzioni ricevono come argomento una riga dei dati del file CSV. Se, per esempio, viene passata la seguente riga:

'200-01-01,-375,BNFC,400401,IT001'

- la funzione ***row_parsing***() restituisce:

  ('BNFC', 1)

  cioè una **tupla** contenente il codice bancario e il numero 1 che rappresenta semplicemente la presenza del codice;

- la funzione **row_parsing_ext**() restituisce:

  ((2000, '400401'), -375.0)

  in cui il **primo elemento della tupla** è a sua volta **un'altra tupla** composta dall'anno dell'operazione e dal numero di conto.

Per ottenere un conteggio degli elementi per codice possiamo usare la prima funzione *row_parsing*() con *map* come indicato nel codice seguente:

```
import module_parsing as mp
rdd=sc.textFile('dati_bancari_annuali.csv')
ris = rdd.map(mp.row_parsing).countByKey()
```

Ottenendo come risultato una struttura dati Python (dizionario) di tipo chiave/valore con il seguente contenuto:

```
defaultdict(<class 'int'>, {'BNFC': 1241, 'VERS': 1315, 'BODA': 1313, 'PAGO': 1277})
```

Il risultato evidenzia 1.241 effettuazioni di bonifici, 1.315 versamenti in contanti, 1.313 bonifici ricevuti e 1.277 pagamenti con bancomat.

Obiettivo 4: calcolare il saldo di ogni conto correnti suddiviso per anni.

Con la seconda funzione del modulo, *row_parsing_ext*(), possiamo procedere a un saldo di ogni conto distinto per anni. Procediamo, quindi, a passarla ad un *map* e a richiamare il metodo *reduceByKey*() come indicato nel codice seguente:

```
saldi_annuali=rdd.map(mp.row_parsing_ext).reduceByKey(lambda a,b: a+b)
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il nuovo RDD ottenuto, *saldi_annuali*, potrà essere interrogato ulteriormente. Per esempio, per ottenere una lista dei saldi annuali del conto numero 600321 è possibile richiamare la **collect**() nel seguente modo:

```
from pyspark import SparkContext
import module_parsing as mp

sc = SparkContext('local','Esempio')

rdd=sc.textFile('dati_bancari_annuali.csv')
saldi_annuali=rdd.map(mp.row_parsing_ext).reduceByKey(lambda a,b: a+b)
ris = saldi_annuali.filter(lambda x: x[0][1]=='600321').sortByKey().collect()
print(ris)

sc.stop()
```

```
[((2000, '600321'), 9740.0), ((2001, '600321'), 6885.0), ((2002, '600321'), 325
45.0), ((2003, '600321'), 18070.0), ((2004, '600321'), 6815.0), ((2005, '60032
1'), 33600.0), ((2006, '600321'), 7800.0), ((2007, '600321'), 10050.0), ((2008,
'600321'), 7625.0), ((2009, '600321'), 21010.0)]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

6. DataFrame

Un **DataFrame** è una **collezione immutabile di dati elaborabile in parallelo su più nodi** di un *cluster*. La struttura di un DataFrame ricorda molto le tabelle di un *database relazionale* o i *fogli di calcolo* (l'idea di fondo è molto simile ai DataFrame di Pandas).

Prima di procedere con gli script PySpark è necessario considerare un aspetto. Mentre per gli **RDD** si utilizzava lo ***SparkContext***, per i **DataFrame** è necessario riferirsi ad un oggetto più esteso che contiene lo SparkContext stesso, la ***SparkSession.***

La maggior parte degli oggetti che verranno utilizzati (tra cui *SparkSession* e *DataFrame*) fa parte del package *pyspark.sql* in cui consiste il *core* di *SparkSQL.*

Esistono diversi modi per istanziare un DataFrame. Uno dei più comuni è quello di importare i dati presenti in un file CSV come quello riportato nell'esempio seguente.

```
nome;cognome;altezza;peso;nascita;sesso
Giulio;Rossi;1.84;89;1994-08-12;M
Alessia;Verdi;1.62;53;1993-03-07;F
Enea;Bianchi;1.78;73;1996-05-06;M
Paola;Neri;1.74;61;1999-09-30;F
Ilenia;gialli;1.70;70;1998-12-27;F
Valerio;Verdi;1.88;88;1996-05-06;M
```

Nel file CSV precedente i dati sono separati da punto e virgola e la prima riga è di intestazione e sarà utilizzata da SparkSQL per nominare le **colonne** del **DataFrame**.

Il codice per importare il file CSV il seguente:

```
import pyspark
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

df = spark.read.csv('persone.csv', sep=';',inferSchema=True,header = True)
```

Con il comando precedente si ottiene un **oggetto *df*** che rappresenta un DataFrame di classe:

pyspark.sql.dataframe.DataFrame

Se si fa riferimento ad una **cartella** invece che ad un file, invece, i contenuti di tutti i file al suo interno saranno disponibili nel DataFrame.

Per visualizzare il contenuto di un DataFrame è possibile utilizzare il metodo **show**() che genera un output puramente di debug, ma evidenziato da trattini che restituiscono l'aspetto di una tabella come mostrato nell'esempio seguente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import pyspark
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

df = spark.read.csv('persone.csv', sep=';',inferSchema=True,header = True)
df.show()
```

```
+-------+-------+-------+----+----------+-----+
|   nome|cognome|altezza|peso|   nascita|sesso|
+-------+-------+-------+----+----------+-----+
| Giulio|  Rossi|   1.84|  89|1994-08-12|    M|
|Alessia|  Verdi|   1.62|  53|1993-03-07|    F|
|   Enea|Bianchi|   1.78|  73|1996-05-06|    M|
|  Paola|   Neri|   1.74|  61|1999-09-30|    F|
| Ilenia| gialli|    1.7|  70|1998-12-27|    F|
|Valerio|  Verdi|   1.88|  88|1996-05-06|    M|
+-------+-------+-------+----+----------+-----+
```

Analizziamo ora ciascuno degli argomenti forniti all'istruzione:

spark.read.csv('persone.csv', sep=';', inferSchema=True, header=True)

- *persone.csv* è il nome del file;
- *sep* indica che il punto e virgola (;) è il separatore dei campi;
- *inferSchema* impostato a *True* indica che PySpark deve intuire da solo il tipo di dato presente in ogni campo;
- *header* a *True* specifica che i dati alla prima riga saranno da intendersi come intestazione delle colonne.

Per verificare il tipo di dato «intuito» dall'istruzione *spark.read.csv* è possibile visualizzare l'attributo **dtypes.**

```
df.dtypes
```

```
[('nome', 'string'),
 ('cognome', 'string'),
 ('altezza', 'double'),
 ('peso', 'int'),
 ('nascita', 'string'),
 ('sesso', 'string')]
```

I campi sono stati così interpretati:

- *nome*, *cognome* e *sesso* come stringhe;
- *altezza* come double;
- *peso* come intero;
- *nascita* come stringa.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

È importante sottolineare che RDD e DataFrame corrispondono a due generazioni diverse di PySpark. Nonostante questo, esiste un modo molto pratico per generare un DataFrame da un RDD con il metodo **createDataFrame**(). Si crea, quindi, una lista Python contenente un certo numero di tuple simili a quelle del file CSV come riportato nel codice seguente:

```
dati = [('Giulio','Rossi',1.84,89,'1994-08-12','M'),
        ('Alessia','Verdi',1.62,53,'1993-03-07','F'),
        ('Enea','Bianchi',1.78,73,'1996-05-06','M')]
print(dati)
```

```
[('Giulio', 'Rossi', 1.84, 89, '1994-08-12', 'M'), ('Alessia', 'Verdi', 1.62, 5
3, '1993-03-07', 'F'), ('Enea', 'Bianchi', 1.78, 73, '1996-05-06', 'M')]
```

Si chiama il metodo **parallelize**() ottenendo una *RDD* passando per lo *SparkContext* come riportato nel codice seguente:

```
sc = spark.sparkContext
rdd=sc.parallelize(dati)
print(rdd.collect())
```

```
[('Giulio', 'Rossi', 1.84, 89, '1994-08-12', 'M'), ('Alessia', 'Verdi', 1.62, 5
3, '1993-03-07', 'F'), ('Enea', 'Bianchi', 1.78, 73, '1996-05-06', 'M')]
```

Successivamente si invoca **createDataFrame**() per convertirlo in un DataFrame come riportato nel codice seguente:

```
df_da_rdd=spark.createDataFrame
(rdd, ['nome','cognome','altezza','peso','nascita','sesso'])
```

Si sottolinea che passando un **argomento numerico** a **show**() invocato su un *DataFrame* è possibile richiedere che venga restituito un determinato numero di righe piuttosto che tutte le righe dell'intero DataFrame.

Nel codice seguente si richiama due volte il metodo show(); il primo senza parametri, il secondo con il parametro 1.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
df.show()
df.show(1)
```

```
+-------+-------+-------+----+----------+-----+
|   nome|cognome|altezza|peso|   nascita|sesso|
+-------+-------+-------+----+----------+-----+
| Giulio|  Rossi|   1.84|  89|1994-08-12|    M|
|Alessia|  Verdi|   1.62|  53|1993-03-07|    F|
|   Enea|Bianchi|   1.78|  73|1996-05-06|    M|
+-------+-------+-------+----+----------+-----+

+------+-------+-------+----+----------+-----+
|  nome|cognome|altezza|peso|   nascita|sesso|
+------+-------+-------+----+----------+-----+
|Giulio|  Rossi|   1.84|  89|1994-08-12|    M|
+------+-------+-------+----+----------+-----+
only showing top 1 row
```

Si riportano, ora, i metodi di ispezione di un DataFrame

- **count**() – conta le righe del DataFrame;
- **first**() – restituisce la prima riga di un DataFrame;
- **take**() – restituisce un certo numero di righe di un DataFrame indicato dall'argomento di tipo intero;
- **head**() – restituisce un certo numero di righe di DataFrame;
- **describe**() – esegue una serie di misure statistiche sul DataFrame (conteggio, media,deviazione standard, minimo e massimo);
- **printSchema**() – stampa la struttura del DataFrame in modo piuttosto leggibile.

Si riportano alcuni esempi riferiti al DataFrame **df** istanziato in precedenza.

```
print("Count: ",df.count())
print()
print("First: ",df.first())
print()
print("Take 2: ",df.take(2))
print()
print("Head 2: ",df.head(2))
```

```
Count:  6

First:  Row(nome='Giulio', cognome='Rossi', altezza=1.84, peso=89, nascita='199
4-08-12', sesso='M')

Take 2:  [Row(nome='Giulio', cognome='Rossi', altezza=1.84, peso=89, nascita='1
994-08-12', sesso='M'), Row(nome='Alessia', cognome='Verdi', altezza=1.62, peso
=53, nascita='1993-03-07', sesso='F')]

Head 2:  [Row(nome='Giulio', cognome='Rossi', altezza=1.84, peso=89, nascita='1
994-08-12', sesso='M'), Row(nome='Alessia', cognome='Verdi', altezza=1.62, peso
=53, nascita='1993-03-07', sesso='F')]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
print("Describe: ",df.describe().show())
print()
print("Print Schema: ",df.printSchema())
```

```
+-------+-------+-------+------------------+------------------+----------+-----+
|summary|   nome|cognome|           altezza|              peso|   nascita|sesso|
+-------+-------+-------+------------------+------------------+----------+-----+
|  count|      6|      6|                 6|                 6|         6|    6|
|   mean|   null|   null| 1.7599999999999998| 72.33333333333333|      null| null|
| stddev|   null|   null|0.09465727652959384|14.361986863476329|      null| null|
|    min|Alessia|Bianchi|              1.62|                53|1993-03-07|    F|
|    max|Valerio| gialli|              1.88|                89|1999-09-30|    M|
+-------+-------+-------+------------------+------------------+----------+-----+

Describe:  None

root
 |-- nome: string (nullable = true)
 |-- cognome: string (nullable = true)
 |-- altezza: double (nullable = true)
 |-- peso: integer (nullable = true)
 |-- nascita: string (nullable = true)
 |-- sesso: string (nullable = true)

Print Schema:  None
```

Osservando il comportamento di *first, take* e *head* si può notare che i risultati forniti sono **oggetti Row**. **Row** è una classe PySpark ***pyspark.sql.types.Row*** che rappresenta un'intera riga di un DataFrame e svolge un importante ruolo per l'estrazione di risultati.

Considerando il risultato di *first* è possibile notare che si possono estrarne singoli valori indicando il nome del campo come indicato nel codice seguente.

```
print("Nome: ", df.first().nome)
print("Cognome: ", df.first().cognome)
print("Peso: ", df.first().peso)
```

```
Nome:  Giulio
Cognome:  Rossi
Peso:  89
```

Un aspetto importante è che, nonostante Row sia parte del framework PySpark, questi dati ottenuti sono normali numeri e stringhe. In questo modo, sono stati, quindi, totalmente estratti dal contesto di PySpark e si può proseguirne l'elaborazione con codice Python. *take* e *head*, invece, sono predisposti per la restituzione di una moltitudine di oggetti. Pertanto, essi restituiscono **una lista di oggetti Row** riconoscibile dalla presenza delle parentesi quadre. Per estrarne singoli valori è necessario prima indicare la posizione e poi il nome del campo come riportato nel codice seguente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
print("Take 2: ",df.take(2))
print()
print(df.take(2)[1].peso)
print(df.take(2)[0].cognome)
```

```
Take 2:  [Row(nome='Giulio', cognome='Rossi', altezza=1.84, peso=89,
nascita='1994-08-12', sesso='M'), Row(nome='Alessia', cognome='Verd
i', altezza=1.62, peso=53, nascita='1993-03-07', sesso='F')]

53
Rossi
```

6.1 Metodi per elaborare dati

Come avviene in SQL, con i DataFrame è possibile svolgere due operazioni molto importanti: **filtro** e **raggruppamento**. Con il **filtro** è possibile richiedere a PySpark di restituire solo le righe corrispondenti a determinate caratteristiche. Il filtro viene eseguito mediante il metodo **filter**() al quale verrà passato come argomento una **clausola booleana** che specifica quali condizioni una riga debba rispettare per essere ammessa tra i risultati. Nell'esempio riportato di seguito, il DataFrame **df** contiene tre donne (filtro sul sesso).

```
df.filter(df['sesso']=='F').show()
```

```
+-------+-------+-------+----+----------+-----+
|   nome|cognome|altezza|peso|   nascita|sesso|
+-------+-------+-------+----+----------+-----+
|Alessia|  Verdi|   1.62|  53|1993-03-07|    F|
|  Paola|   Neri|   1.74|  61|1999-09-30|    F|
| Ilenia| gialli|    1.7|  70|1998-12-27|    F|
+-------+-------+-------+----+----------+-----+
```

La condizione specificata è:

df['sesso']=='F'

Si noti che il nome del campo di interesse viene specificato all'interno delle parentesi quadre, mentre gli operatori di confronto utilizzati sono quelli soliti della programmazione.

In *alternativa alle parentesi quadre* è possibile utilizzare il punto (.) per separare il nome del DataFrame e quello del campo. Infatti, la seguente espressione restituisce il medesimo risultato ottenuto precedentemente:

```
df.filter(df.sesso=='F').show()
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il **groupBy**() permette, invece, di raggruppare dei dati e richiede di specificare i campi in base ai quali aggregarli. Dopo il groupBy() si possono invocare funzioni per le operazioni più standard come il **count**() per il conteggio. Nel codice seguente si contano i numeri dei maschi e delle femmine presenti nel DataFrame (raggruppamento per sesso e conteggio degli elementi dei gruppi).

```
df.groupBy('sesso').count().show()
```

```
+-----+-----+
|sesso|count|
+-----+-----+
|    F|    3|
|    M|    3|
+-----+-----+
```

Il risultato del **count**() è, a sua volta, un *DataFrame*. È questo il motivo per il quale è stato possibile invocare su di esso il metodo **show**().

Quando è necessario personalizzare maggiormente le aggregazioni da svolgere, è possibile utilizzare il metodo **agg**() che si occuperà di specificare esattamente cosa si vuole eseguire sui gruppi di dati composti dal **groupBy**(). A tal proposito, si ricorre ad un **package di funzioni** che per Spark mette a disposizione di nome *pyspark.sql.functions.* Per importare tale package basterà scrivere il seguente codice:

from pyspark.sql import functions as F

Le funzioni in messe a disposizione dal package sono molte tra cui funzioni matematiche, trigonometriche, di conversione di formato, per le stringhe, l'elaborazione di date, hashing, statistica, ecc.

Ad esempio, si vuole trovare quali siano il **peso medio e l'altezza media dei maschi e femmine** nel DataFrame con il seguente codice:

```
from pyspark.sql import functions as F

df.groupBy('sesso').agg(F.mean('peso'), F.mean('altezza')).show()
```

```
+-----+------------------+------------------+
|sesso|         avg(peso)|      avg(altezza)|
+-----+------------------+------------------+
|    F|61.333333333333336|1.6866666666666668|
|    M| 83.33333333333333|1.8333333333333333|
+-----+------------------+------------------+
```

Il metodo **agg**() ha lavorato sul risultato del *groupBy*() e ha ricevuto due argomenti:

1. F.mean('peso') - calcola il peso medio degli appartenenti ad ogni gruppo;
2. F.mean('altezza') - restituisce l'altezza media per i singoli gruppi.

Osservando la tabella si notano due aspetti migliorabili:

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. i numeri dovrebbero essere **arrotondati** per poter essere più leggibili;
2. *avg(peso)* e *avg(altezza)* non sono certo nomi di campo facilmente utilizzabili.

Per tale ragione, si utilizzerà la funzione di arrotondamento **round**() e il metodo **alias**() per la sostituzione del nome di un campo come riportato nel codice seguente:

```
from pyspark.sql import functions as F

df.groupBy('sesso').agg(F.round(F.mean('peso'),2).alias('peso medio'),
                        F.round(F.mean('altezza'),2).alias('altezza media')).show()
```

```
+-----+----------+-------------+
|sesso|peso medio|altezza media|
+-----+----------+-------------+
|    F|     61.33|         1.69|
|    M|     83.33|         1.83|
+-----+----------+-------------+
```

Dall'analisi del codice precedente, si nota che è stato necessario **annidare alcune funzioni**.

Con l'invocazione di *F.round(F.mean('altezza'),2)* è stata calcolata la **media del campo peso** e, successivamente, è stata arrotondata con round a due cifre decimali. Infine, con *alias('peso medio')* è stato applicato il nuovo nome del campo alla colonna risultato. Le stesse operazioni sono state svolte per il campo altezza.

6.2 Presentare i dati

Dopo aver eseguito le operazioni di filtro e raggruppamento è necessario **presentare i dati nel modo migliore**. I principali metodi per tale finalità sono:

- **orderBy**() per l'ordinamento dei dati. Restituisce un altro *DataFrame* ordinato in base al campo specificato;
- **limit**() per specificare il numero di dati che devono essere espressi;
- **select**() per definire la proiezione dei dati ovvero il modo in cui devono essere restituiti nel risultato finale.

Nell'esempio che segue, si richiede che le righe siano ordinate per altezza.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
df.orderBy('altezza').show()
```

```
+-------+-------+-------+----+----------+-----+
|   nome|cognome|altezza|peso|   nascita|sesso|
+-------+-------+-------+----+----------+-----+
|Alessia|  Verdi|   1.62|  53|1993-03-07|    F|
| Ilenia| gialli|    1.7|  70|1998-12-27|    F|
|  Paola|   Neri|   1.74|  61|1999-09-30|    F|
|   Enea|Bianchi|   1.78|  73|1996-05-06|    M|
| Giulio|  Rossi|   1.84|  89|1994-08-12|    M|
|Valerio|  Verdi|   1.88|  88|1996-05-06|    M|
+-------+-------+-------+----+----------+-----+
```

L'ordinamento di default è crescente, ma applicando l'argomento **ascending** impostato a *False* si ottengono i dati ordinati per altezza decrescente come riportato nel codice seguente:

```
df.orderBy('altezza', ascending=False).show()
```

```
+-------+-------+-------+----+----------+-----+
|   nome|cognome|altezza|peso|   nascita|sesso|
+-------+-------+-------+----+----------+-----+
|Valerio|  Verdi|   1.88|  88|1996-05-06|    M|
| Giulio|  Rossi|   1.84|  89|1994-08-12|    M|
|   Enea|Bianchi|   1.78|  73|1996-05-06|    M|
|  Paola|   Neri|   1.74|  61|1999-09-30|    F|
| Ilenia| gialli|    1.7|  70|1998-12-27|    F|
|Alessia|  Verdi|   1.62|  53|1993-03-07|    F|
+-------+-------+-------+----+----------+-----+
```

Il metodo **limit**() permette di richiedere il numero di righe che si desidera visualizzare. Esso funziona molto bene insieme *orderBy*(). Infatti, se si volessero avere solo i dati relativi al soggetto più alto, sarebbe possibile ottenerlo nel seguente modo:

```
df.orderBy('altezza', ascending=False).limit(1).show()
```

```
+-------+-------+-------+----+----------+-----+
|   nome|cognome|altezza|peso|   nascita|sesso|
+-------+-------+-------+----+----------+-----+
|Valerio|  Verdi|   1.88|  88|1996-05-06|    M|
+-------+-------+-------+----+----------+-----+
```

Un utilizzo simile di **limit**() potrebbe ricordare quello di *first*() o, in generale, dei metodi come *take*() e *head*(), ma esiste una **differenza fondamentale**:

- **limit**() restituisce un DataFrame, quindi, appartiene alle operazioni di trasformazione di SparkSQL;

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- *first*(), *take*() e *head*(), invece, restituiscono normali oggetti Python di tipo **Row eventualmente raggruppati in liste**.

Il metodo **select**() si occupa della produzione di risultati, ovvero con i suoi argomenti indica quali campi si vogliono mostrare e quali funzioni utilizzare per trasformarne i valori prima di restituirli al chiamante.

Rimanendo sull'esempio del DataFrame ***df*** è possibile mostrare i risultati del raggruppamento visto precedentemente. Potremmo, però, non essere interessati a mostrare il singolo carattere 'M' o 'F', ma in fase di produzione sostituirlo con le versioni estese 'MASCHIO' o 'FEMMINA'. In tal caso, è possibile utilizzare il metodo **select**() e la funzione **when**() del package *pyspark* come riportato nel codice seguente.

```
df.groupBy('sesso')\
.agg(F.min('peso').alias('peso minimo'),F.max('peso').alias('peso massimo'))\
.select(F.when(df.sesso=='M','MASCHIO').otherwise('FEMMINA').\
        alias('sesso'), 'peso minimo', 'peso massimo').show()
```

```
+-------+-----------+------------+
|  sesso|peso minimo|peso massimo|
+-------+-----------+------------+
|FEMMINA|         53|          70|
|MASCHIO|         73|          89|
+-------+-----------+------------+
```

Si notano alcuni aspetti:

- la funzione **when**() verifica la condizione *df.sesso='M'* e in tal caso propone come valore 'MASCHIO';
- in coppia con **when**() c'è **otherwise**() che rappresenta l'alternativa al risultato positivo del confronto eseguite da ***when***();
- più invocazioni **when**() possono essere concatenate effettuando più confronti alternativi fra loro.

L'invocazione di **select**() ha lavorato sul risultato del **groupBy**() e, nel nominare quali campi saranno prodotti in output, deve utilizzare i nomi impostati con il metodo **alias**().

A titolo informativo, in PySpark, è possibile utilizzare anche il linguaggio SQL per effettuare interrogazioni in uno stile identico a quello dei database relazionali. Tale modalità permette da un lato di mettere i programmatori totalmente a proprio agio poiché richiama un modello di lavoro molto tradizionale ed è, inoltre, totalmente compatibile con il flusso di lavoro di SparkSQL in quanto offre DataFrame come risultato delle query.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Maggi G. (2020). Data science con Python. Edizioni LSWR.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).